

Robot Baseball

Jakob Sverre Alexandersen
Jane Street Submission

October 14, 2025

The Artificial Automaton Athletics Association (Quad-A) is at it again, to compete with postseason baseball they are developing a Robot Baseball competition. Games are composed of a series of independent at-bats in which the batter is trying to maximize expected score and the pitcher is trying to minimize expected score.

An at-bat is a series of pitches with a running count of balls and strikes, both starting at zero. For each pitch, the pitcher decides whether to throw a ball or strike, and the batter decides whether to wait or swing; these decisions are made secretly and simultaneously. The results of these choices are as follows.

- If the pitcher throws a ball and the batter waits, the count of balls is incremented by 1.
- If the pitcher throws a strike and the batter waits, the count of strikes is incremented by 1.
- If the pitcher throws a ball and the batter swings, the count of strikes is incremented by 1.
- If the pitcher throws a strike and the batter swings, with probability p the batter hits a home run and with probability $1 - p$ the count of strikes is incremented by 1.

An at-bat ends when either:

- The count of balls reaches 4, in which case the batter receives 1 point.
- The count of strikes reaches 3, in which case the batter receives 0 points.
- The batter hits a home run, in which case the batter receives 4 points.

By varying the size of the strike zone, Quad-A can adjust the value p , the probability a pitched strike that is swung at results in a home run. They have found that viewers are most excited by at-bats that reach a full count, that is, the at-bats that reach the state of three balls and two strikes. Let q be the probability of at-bats reaching full count; q is dependent on p . Assume the batter and pitcher are both using optimal mixed strategies and Quad-A has chosen the p that maximizes q . Find this q , the maximal probability at-bats reach full count, to ten decimal places.

1 Model and objective

States are (b, s) with $b \in \{0, 1, 2, 3\}$, $s \in \{0, 1, 2\}$. Terminals:

- If $b = 4$ (4 balls) the at-bat ends (walk), it did not reach full count unless the state $(3, 2)$ was previously visited; for our forward recursion, transitions into an immediate terminal that is not $(3, 2)$ have value 0 for the indicator “did we reach $(3, 2)$ at any time?”.
- If $s = 3$ the at-bat ends (strikeout), value 0
- If we ever reach $(3, 2)$ that counts as “full count reached”, we treat that state as absorbing with value 1 for the probability q we’re computing

On each pitch, pitcher chooses $P \in \{\text{Ball}, \text{Strike}\}$; batter chooses $A \in \{\text{Wait}, \text{Swing}\}$. The deterministic transitions:

- $(\text{Ball}, \text{Wait}) \rightarrow (b + 1, s)$ (or terminal if $b + 1 = 4$)
- $(\text{Strike}, \text{Wait}) \rightarrow (b, s + 1)$ (or terminal if $s + 1 = 3$)
- $(\text{Ball}, \text{Swing}) \rightarrow (b, s + 1)$ (a swing ball counts as a strike)
- $(\text{Strike}, \text{Swing}) \rightarrow$ with probability p home run (terminal; does not give full count unless we had already been at $(3, 2)$), in our forward recursion treat this outcome as terminal with value 0, and with probability $1 - p$ it becomes $(b, s + 1)$ (or terminal if $s + 1 = 3$)

We are computing the probability, under optimal (minimax) play, that the process ever visits $(3, 2)$. This is a zero-sum game: pitcher aims to minimize that probability, batter aims to maximize it.

2 Bellman / 2×2 game at each state

Let $Q_{b,s}$ be the value (probability of reaching full count from state (b, s)) under optimal play. For each nonterminal state (b, s) the immediate 2×2 payoff matrix (pitcher rows Ball, Strike; batter columns Wait, Swing) has entries equal to the *probability of eventually reaching full count* resulting from that pitch (i.e. the $\mathbb{E}Q$ of the next state):

Denote next-state-vvalue function $\text{next}(b', s') =$ value for that state, but if a transition is to an immediate terminal that is not $(3, 2)$, that entry is 0; if the transition is to $(3, 2)$, that entry is 1.

Concretely (for a nonterminal state (b, s)):

- $$M_{B,W} = \text{value after (Ball, Wait)} = \begin{cases} 1 & \text{if } (b + 1, s) = (3, 2) \\ 0 & \text{if } b + 1 = 4 \text{ and } (b + 1, s) \neq (3, 2) \\ Q_{b+1,s} & \text{otherwise} \end{cases}$$
- $M_{B,S_w} = \text{value after (Ball, Swing)} =$ similarly uses $(b, s + 1)$ (or 0 if $s + 1 = 3$ and not $(3, 2)$, or 1 if $(b, s + 1) = (3, 2)$)
- $M_{S,W} = \text{value after (Strike, Wait)} =$ same as for $(b, s + 1)$
- $M_{S,S_w} = \text{value after (Strike, Swing)} = p \cdot 0 + (1 - p) \cdot Q_{b,s+1} = (1 - p) \cdot Q_{b,s+1}$, except check terminal conditions if $s + 1 = 3$ (then that term is 0 or 1 as above).

Given this 2×2 matrix $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ (rows Ball / Strike; cols Wait / Swing), the value V of the zero-sum game (pitcher minimizing the payoff, batter maximizing) is the standard matrix-game value. Algo to compute V robustly:

1. Compute pure-strategy values: row minima and column maxima.
2. If a pure saddle exists ($\max\min = \min\max$), V is that value.
3. Otherwise the unique interior mixed strategy exists: let batter mix with probability x on Wait and $1 - x$ on Swing to make the pitcher indifferent. Solve

$$a \cdot a + (1 - x) \cdot b = x \cdot c + (1 - x) \cdot d,$$

solve for $x \in [0, 1]$. Then plug back to get $V = x \cdot a + (1 - x) \cdot b$. (Alternatively mix the pitcher, either way is equivalent.)

Because the state graph is acyclic (balls and strikes only increase), we can compute values by backward recursion: evaluate $Q_{b,s}$ in decreasing order of $(b + s)$ (i.e., compute states that lead only to terminals or already-computed states).

3 Numerical procedure and reproducible code

Run the code in `main.py` to compute $q(p) = Q_{0,0}$ for a given p and to find the p maximizing q . It uses exact backward recursion over the 4×3 grid of states and solves the 2×2 game by checking pure strategies first and otherwise solving the interior-mix equality.

$q_{\text{opt}} = 0.2959679934$